Real-world (act out a canary release) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Hint 1 — The idea, and the exact pieces

The whole trick of a canary release is that the *routing decision* happens independently for every single request — you're not splitting your list of test requests into "the first 20%" and "the rest," which would just be an arbitrary chunk, not a genuine random sample. `random.random() < 0.2` called once per request gives you a fair, independent coin flip each time, so the canary group ends up looking statistically like a real slice of your traffic, not whatever happened to be first in the file.

You need somewhere to record the outcome of each request, tagged with which version handled it, so you can group and compare afterward — a list of small dicts is the natural fit here, the same shape you've used for the version log.

The exact pieces:

- `import random` — `random.random()` returns a float in `[0.0, 1.0)`.
- A loop over your test requests: `for request in test_requests:`.
- A results list: append `{"version": "v1" or "v2", "score": ..., "cost": ...}` for every request, inside the loop.
- After the loop: separate the results by version (two lists, built with a plain `for` loop and `if`, not a comprehension), then `sum(...) / len(...)` for each group's average score and average cost.

Hint 2 — The plan, and almost the whole thing

```
function route_and_score(test_requests, prompts, canary_share) -> list[dict]:
    results = []
    for each request:
        version = "v2" if a random draw is under canary_share, else "v1"
        result = run_eval_suite(prompts[version])
```

```
        record version, score, and a fake cost estimate in results
    return results

function summarize(results, version) -> dict:
    matching = only the entries for this version
    return count, average score, average cost
```

```
# canary_release_practice.py
import random

def route_and_score(
    test_requests: list, prompts: dict, canary_share: float = 0.2
) -> list[dict]:
    results = []
    for request in test_requests:
        if random.random() < canary_share:
            version = "v2"
        else:
            version = "v1"
        eval_result = run_eval_suite(prompts[version])
        results.append({
            "version": version,
            "score": eval_result["score"],
            "cost": len(prompts[version]) * 0.0001,  # stand-in cost estimate
        })
    return results

def summarize(results: list[dict], version: str) -> dict:
    matching = []
    for r in results:
        if r["version"] == version:
            matching.append(r)

    scores = []
    costs = []
    for r in matching:
        scores.append(r["score"])
        costs.append(r["cost"])

    return {
        "version": version,
        "count": len(matching),
        "avg_score": sum(scores) / len(scores),
        "avg_cost": sum(costs) / len(costs),
    }
```

Now call `summarize` for both `"v1"` and `"v2"` and print both, then compare against the Solution.

Full solution: Show me the solution